```
    if cmpShops((dist, node.shop), (nnDist, nn)) < 0
then            #5
      (nnDist, nn) ← (dist, node.shop)
    if compare(location, node) < 0
then                                          #6
      closeBranch ← node.left
      farBranch ← node.right
    else


       #7
      closeBranch ← node.right
      farBranch ← node.left
    (nnDist, nn) ← fNN(closeBranch, location, cmpShops,
(nnDist, nn)) #8
    if splitDistance(location, node) < nnDist
then                    #9
      (nnDist, nn) ← fNN(farBranch, location, cmpShops,
(nnDist, nn))          #10
    return (nnDist,
nn)                                                          #11
```

#1 Find the closest point to a given target. In this case, we pass `cmpShops` method to compare two shops and return which one is best. We assume the return value uses the standard logic for the compare functions: `-1` means the first argument is smaller, `1` the second is smaller, and `0` they are equal.

#2 If node is `null`, we are traversing an empty tree, so the nearest neighbor can’t change with respect to what we have already found.

#3 Otherwise, we have 3 tasks: check if current node is better than previously found NN, traverse the branch on the same side of the split with respect to the target point, and check if we can prune the other branch (or traverse it as well).